Contents

E-Level Engineer Interview Guide

Internal interview — two candidates, three-person panel.

Role	Person
Engineering Lead	Joseph Aruja
Product Owner	(PO)
Senior Engineer (E-Level)	(peer interviewer)

Project context for interviewers: the team owns the **ODP** (Original Data Product) and **FDP** (Foundation Data Product) layers in BigQuery. The team also produces **CDPs** (Consumable Data Products) which depend on FDPs maintained by adjacent teams. The candidates being interviewed are internal, so assume they have organisational context but not necessarily the depth on the patterns below.

How to use this guide

- **Read the guide end-to-end before the interview.** Take 30 minutes. Calibrate on what "great", "good", and "weak" look like for each question, so your scoring is consistent.
- **Pre-agree question split.** Joseph leads the architecture-heavy ones; the peer engineer leads the cloud and CI/CD specifics; the PO leads the conversational / collaboration probes. We rotate question ownership but only one interviewer asks each question — the others listen and take notes.
- **Score in real time on the score sheet at the bottom of this guide**, not after.
- **Follow up.** The follow-up probes matter as much as the original question. They expose whether the candidate has built the thing or only read about it.
- **Avoid leading.** If a candidate says "we'd use Composer", do not say "and what about Cloud Run?". Ask "what made you choose Composer?" and let them either justify or unravel.

- **Take five minutes between candidates** to debrief on first impressions before they fade.

  Calibration note — score the concept, not the vocabulary.

  Candidates may use different terminology to describe the same idea — "pre-stage table" vs "ODP", "fan-in transform" vs "JOIN pattern", "audit log" vs "audit trail", "side outputs" vs "tagged outputs". The terms below reflect *our* internal vocabulary. Score on whether the candidate has the *concept* right, not whether they say the words we use. If a candidate fluently describes the right pattern under a different name, that is a strong signal, not a weak one. Probe to confirm shared understanding ("when you say *pre-stage*, do you mean an untransformed mirror of the source in BigQuery?") rather than correcting their language.

  Similarly, where the guide quotes specific numbers — Composer at ~£300/month, "<50M rows/day" thresholds — treat them as ballpark heuristics for your own calibration, not as facts a candidate must reproduce. A candidate citing a sensible but different figure is fine.

Format and timing — 60 minutes per candidate

We have one hour per candidate. Six primary questions, ~7 minutes each, plus 10 minutes for the candidate to ask us questions.

Block	Duration	Lead
Welcome + intro	3 min	PO
Q1 — ODP/FDP pipeline from fixed-width GCS file (incl. "no golden path" probe)	9 min	Joseph
Q2 — Data remediation steps	7 min	PO
Q3 — Prevent and recover from duplication in dbt FDP (Q3 + Q7 merged)	7 min	Peer engineer
Q5 — dbt auth dev vs prod	5 min	Peer engineer
Q8 — Scenario: validating a bespoke GDW CDP in production (two-part)	8 min	Joseph
Q9 — CI/CD for dbt or Dataflow	9 min	Peer engineer
Candidate questions for us	10 min	All
Wrap	2 min	Joseph

Total: 60 minutes per candidate.

Dropped from the live interview (kept in this guide as a deeper-dive bank — useful follow-up probes if a candidate cruises through the primary questions, or for the second-round written exercise if we run one):

- **Q4 — Methods to run Dataflow.** Can probe via Q1 follow-up ("how would you actually launch this Dataflow job?").
- **Q6 — BigQuery storage optimisation.** Least senior signal; better as a written-exercise question.

Q7 (FDP duplication after accidental re-run) is now merged into the expanded Q3 — the candidate is asked to cover prevention *and* recovery. Strong candidates do both naturally.

If a candidate sails through and you have time in hand, use Q4 or Q6 from the bank to probe deeper.

Section A — Data Engineering Examples

These two questions test whether the candidate can think architecturally about a real pipeline, not just describe components in isolation. We expect them to draw / talk through a flow.

Q1. How would you set up data pipelines to build out an ODP and FDP layer, starting with a fixed-width file landing in a GCS bucket?

What we're testing: end-to-end architectural thinking; understanding of separation of concerns; awareness of operational realities (audit, retries, validation, observability).

Give the candidate a whiteboard / sketch sheet. Encourage them to draw.

A great answer covers all of these:

1. **Trigger.** GCS notification → Pub/Sub topic → an orchestrator. Names options and trade-offs: Composer/Airflow for teams with many DAGs and complex dependency graphs (real-money baseline cost, typically £300–£450 / month for a minimal Composer 2 environment in EU regions); Cloud Functions + Cloud Run Jobs for smaller teams who don't need Airflow's full feature set; Workflows for sequencing without an orchestrator. No single threshold is correct; a candidate justifying their pick on operational grounds is fine.
2. **Landing.** GCS landing bucket with lifecycle rules (move to Coldline after N days). Mentions an `.ok` sentinel file or similar trigger discipline to avoid processing partial uploads.
3. **File parsing.** This is a *fixed-width* file. The candidate should think about parsing the record layout itself — typically driven by a copybook, layout spec, or hard-coded column-position table (e.g., chars 1–10 → `customer_id`, chars 11–50 →

`name`, chars 51–58 → `date_of_birth` in YYYYMMDD). Apache Beam doesn't have a native fixed-width reader, so they'd build a `DoFn` that takes each line from `TextIO` and applies substring offsets. Bonus points if they mention encoding (EBCDIC if it's a mainframe extract, UTF-8/Latin-1 otherwise), trailing-space stripping, and date-format handling.

4. **Envelope handling (if present).** Separately from the fixed-width parsing, mainframe-style files often have an envelope: a `HDR` header line at the top and a `TRL` trailer at the bottom carrying record counts. A great candidate distinguishes the two concerns — the layout tells them how to *parse a record*; the envelope tells them how to *validate the batch*. If the file has no envelope, this step is skipped.
5. **Ingestion pipeline.** Beam on Dataflow (Flex Template) reads the file, parses each record via the layout, validates against a schema, and splits good vs bad records via tagged outputs. Bad records go to a quarantine bucket with audit metadata. Good records load into the **ODP** table in BigQuery — flat, untransformed, one table per source entity, audit-columned (`_loaded_at`, `_run_id`, `_source_system`).
6. **Reconciliation.** The candidate explicitly mentions verifying that `envelope_count = valid + invalid = BigQuery row count` (if there was an envelope). A great candidate stores this in a `reconciliation` or `pipeline_runs` table even when no envelope exists, recording at minimum the loaded row count and Dataflow job ID.
7. **Transformation.** dbt project reads ODP, writes **FDP** tables. Incremental materialisation, partitioning, clustering. Two patterns called out (under whatever names the candidate prefers): **one-to-one** (one ODP source → one FDP target, fires immediately on source load) and **fan-in** (many ODP sources → one FDP target, must wait for all sources). The orchestrator gates the fan-in transformation behind a check that all required sources have loaded for the extract date.
8. **CDP layer.** They acknowledge CDPs sit on top of FDPs and may depend on other teams' FDPs — flag the contract risk and how they'd handle it (versioned views, dbt sources with explicit owners, fail-fast on schema drift).
9. **Observability.** Run IDs propagated through every component (log line, audit row, dbt invocation, BigQuery row), structured logging, cost tracking.
10. **Recovery.** What happens if Dataflow fails mid-job? What happens if dbt errors out? Idempotence — can the pipeline re-run safely?

A good answer covers items 1–7 and 9. They might miss CDP nuance or skip reconciliation, but the core ODP→FDP flow is solid.

An average answer describes the components in order ("file lands, then Dataflow runs, then dbt runs") without separating responsibilities or naming patterns. No mention of audit, retries, idempotence, or reconciliation. Just "it works".

Weak signals:

- Treating dbt as just SQL, with no awareness of materialisation strategies.
- Saying "we'd use Composer" without justification, especially without acknowledging cost.
- Conflating ODP and FDP (e.g., "we'd transform during ingestion") — that's a red flag for a senior data engineer.

- Forgetting that "a fixed-width file" needs deliberate parsing — they should mention column-position-based parsing or HDR/TRL envelopes.
- No mention of error handling.

Follow-up probes (pick 1–2 if time):

- **"What if there's no golden path in place for this — say you're the first team to build a CDP off this CDS, and there's no precedent. Where do you start?"** ← (PO's specific ask: tests whether the candidate can architect from scratch, not just follow a template.)
- "What's the failure mode if the Dataflow job crashes after writing half the rows to BigQuery?"
- "How would you keep CDP consumers safe when an upstream FDP schema changes?"
- "Show me where you'd put audit columns and what's in them."
- "Would you put logic in Beam or in dbt? Why?"

Score 1–5:

- 5 — articulates the full pipeline with separation, audit, reconciliation, and CDP contract awareness; can explain trade-offs.
- 4 — covers ODP/FDP/orchestration cleanly, slight gaps on CDP or reconciliation.
- 3 — describes the components but doesn't separate responsibilities; misses audit/observability.
- 2 — confuses ingestion with transformation, misses fixed-width specifics, no operational thinking.
- 1 — can't articulate a coherent pipeline.

Q2. How would you build data remediation steps into your data pipelines?

What we're testing: real production thinking. Anyone can describe a happy path; remediation is where senior engineers shine.

A great answer covers:

1. **Defines what remediation means.** Replay of failed records, backfill of historical corrections, deletion of erroneous data, schema-evolution handling, late-arriving data, manual one-off fixes.
2. **For replay:** quarantine bucket pattern — invalid records land with metadata (raw record, validation error, run ID, timestamp). A separate "replay" DAG reads quarantine bucket, attempts re-ingestion with optional patches, audits the outcome.
3. **For backfill:** dbt incremental models with a `full_refresh` flag, partition-scoped re-runs (`--vars 'backfill_start=2026-01-01'`), or scheduled small windows to avoid scanning years of data at once. They mention cost implications.
4. **For deletion:** safe-deletion workflow with approval gates (review → hold → delete → archive) rather than `DELETE FROM`. Tombstone rows for audit. Mentions GDPR/PCI/regulatory framing if relevant.

5. **For schema evolution:** strategies for adding columns (safe, default NULL), removing columns (deprecation period), renaming columns (dual-write, then cutover).
6. **For late-arriving data:** out-of-order handling — late records flagged, possibly re-triggering downstream FDP/CDP refreshes.
7. **Auditability:** every remediation action logged to a `data_remediation_events` table or equivalent. Who did what, when, why, against which run ID.
8. **Idempotency:** any remediation pipeline must be safe to re-run.

A good answer covers replay + backfill + auditability with a clear story.

An average answer only describes manual fixes ("we'd write a SQL script") with no automation or audit.

Weak signals:

- "We just delete and reload" — no understanding of audit or regulatory requirements.
- No mention of audit trail for remediation actions.
- Suggesting destructive `DELETE FROM production` casually.
- No idempotency thinking.

Follow-up probes:

- "How would you remediate a single bad customer record that affects a year's worth of FDP?"
- "Who approves a deletion in your pipeline, and how is that recorded?"
- "What's your approach if a column type needs to change retroactively?"
- "How do downstream CDPs find out a backfill is happening?"

Score 1–5:

- 5 — covers replay, backfill, deletion, schema evolution, and audit; mentions regulatory context.
- 4 — covers 3 of those plus audit.
- 3 — covers replay and one of (backfill / deletion).
- 2 — manual-only thinking, no audit.
- 1 — no concept of remediation as a distinct concern.

Section B — Cloud (GCP / Dataflow / dbt / BigQuery)

These test platform-specific knowledge. We expect E-Level engineers to have hands-on opinions about *how*, not just *what*.

Q3. How would you prevent and recover from data duplication in your FDP layer using dbt? (e.g., the upstream pipeline accidentally ran twice)

What we're testing: practical dbt knowledge **plus** operational thinking. Anyone can name the SQL pattern; we want to see how they handle it when it has already

happened.

Note for interviewer: this merges the original Q3 (prevention) and Q7 (recovery from accidental re-run) into one question. Strong candidates will cover both halves; weaker ones will only cover prevention. Probe explicitly for recovery if they don't volunteer it: *"Now imagine it has already happened — how do you fix it without breaking the downstream CDPs?"*

A great answer covers:

1. **Incremental models with `unique_key`.** dbt's `incremental` materialisation with a `unique_key` and `incremental_strategy='merge'` is the primary defence — a merge that updates existing rows rather than appending duplicates.
2. **Deduplication CTEs.** Inside the SQL, a `ROW_NUMBER() OVER (PARTITION BY business_key ORDER BY load_timestamp DESC)` pattern to deduplicate within a batch before insert.
3. **Sources with surrogate keys.** Generating a deterministic hash key (`dbt_utils.generate_surrogate_key`) so re-ingested data produces the same key and `merge` works.
4. **`is_incremental()` guard with predicate.** Filtering source ODP rows by `_loaded_at > (SELECT MAX(_loaded_at) FROM {{ this }})` so the same batch can't be processed twice.
5. **dbt tests for uniqueness.** `unique` and `not_null` tests on the primary key, run after the model materialises, fail fast on duplicates.
6. **Idempotent runs.** A second `dbt run` of the same model with the same source data produces the same target table — by design.

A good answer covers points 1, 2, 5.

An average answer says "I'd add a `unique` test" without explaining the merge strategy or how a duplicate is prevented at write time.

Weak signals:

- Using `append` materialisation without thinking about duplicates.
- "We'd just write a SQL DELETE before insert" — fragile, doesn't scale.
- No mention of dbt tests.

Follow-up probes:

- "Show me the SQL for an incremental model with `unique_key`."
- "What does `merge` actually do under the hood in BigQuery?"
- "Suppose the source has a duplicate already — how does your dbt model handle it?"

Score 1–5: as above; 5 if they show the full picture and can write the merge predicate by hand.

Q4. What methods would you use to run Dataflow, and what are the benefits of each method?

> **Follow-up bank — not in the 60-min slot.** Use this only if a candidate finishes the primary questions with time to spare, or as a written-exercise question for a second round.

What we're testing: awareness of Dataflow's launch patterns and when to pick each.

A great answer names at least three of these and gives trade-offs:

1. **Direct `gcloud dataflow jobs run` from CLI / CI** — for one-off launches; you provide the staged code path. Simple, no template artefact.
2. **Dataflow classic templates** — pre-staged template metadata. Limited parameter types. Largely superseded by Flex Templates.
3. **Dataflow Flex Templates** — packaged as a Docker image with parameter manifest; launched on demand with a JSON parameter set. **Recommended for production.** Benefits: parameterisable, versioned (image tag), reproducible across environments, callable from Airflow / Cloud Workflows / shell.
4. **Direct Python/Java SDK invocation** — `python pipeline.py --runner=DataflowRunner --project=...`. Useful for dev; not used in production.
5. **DirectRunner** — for local dev/test without Dataflow. Free, fast iteration, but doesn't exercise distributed semantics.
6. **Airflow `DataflowFlexTemplateOperator`** — the production launch pattern when DAGs are involved.

A great candidate also mentions **why Flex Templates win** for production: artefact immutability, parameter validation, easier rollback (image tag), and works well with CI/CD (build image once → tag → use across environments).

A good answer covers Flex Templates + DirectRunner + Airflow integration.

An average answer says "we just run it from a CLI" with no mention of templates or image-based deployment.

Weak signals:

- Not knowing about Flex Templates at all.
- Suggesting that the Beam pipeline code lives directly in the Airflow DAG (it doesn't — DAG launches a template).

Follow-up probes:

- "Why a Flex Template over a classic template?"
- "How would you parameterise the Flex Template for prod vs dev?"
- "How do you roll back a bad Dataflow deployment?"

Q5. How would you authenticate with dbt to your project for development, and also when running in production?

What we're testing: secure credentials handling, awareness of Workload Identity Federation, separation of dev vs prod.

A great answer covers:

1. **For development:**
   - `gcloud auth application-default login` for the developer's personal credentials (Application Default Credentials, ADC).
   - Optional **service account impersonation** so the developer runs dbt queries with the SA's permissions (matching production behaviour) without needing the SA's key. Crucially, **dbt-bigquery picks impersonation up from `profiles.yml`**, not from the `gcloud config` setting alone. The profile entry looks like:

     ```yaml
     dev:
       type: bigquery
       method: oauth
       project: my-dev-project
       dataset: dbt_dev_alice
       impersonate_service_account: dbt-dev-sa@my-dev-project.iam.gserviceaccount.co
     ```

     A bonus point if they mention the underlying mechanism: the dev's ADC must have `roles/iam.serviceAccountTokenCreator` on the target SA, and dbt-bigquery exchanges short-lived tokens at query time. (`gcloud config set auth/impersonate_service_account ...` is useful for ad-hoc `gcloud`/`bq` CLI verification but dbt does not read it.)
   - **No JSON keys downloaded to laptops.** Key files are the #1 cause of leaked credentials.
2. **For production:**
   - **Workload Identity Federation** if dbt runs in GitHub Actions (or any external CI) — no long-lived service account keys.
   - **Service account attached to the runtime** if dbt runs inside GCP (Cloud Run, Composer, GKE) — the workload uses the attached SA's identity directly via the metadata server.
   - **Separate SAs for dev vs prod** with least-privilege roles (dbt-dev-sa has read on raw, write on dev datasets; dbt-prod-sa has read on raw, write on prod datasets; no cross-environment access).
   - **profiles.yml** per environment, using OAuth token or impersonation — never embedded keys.
3. **Secret management:** any tokens that must exist (PyPI tokens, Slack webhooks) are stored in Google Secret Manager and retrieved at runtime.

A good answer mentions ADC + service accounts + WIF, possibly without the impersonation detail.

An average answer says "we'd use a service account JSON key" — for production this is technically possible but no longer recommended.

Weak signals:

- Suggesting JSON service account keys in `profiles.yml`.
- Suggesting committing credentials to git.
- Not knowing what WIF is at all.

Follow-up probes:

- "How do you keep a developer from accidentally running dbt against the prod dataset?"
- "Walk me through WIF setup for GitHub Actions → GCP."
- "What if dbt is running inside Composer — how does authentication work?"

Q6. What methods would you use to optimise your storage of data in BigQuery?

> **Follow-up bank — not in the 60-min slot.** Best as a take-home written-exercise question.

What we're testing: breadth of BigQuery operational knowledge — storage and query both.

A great answer covers at least 5 of these:

1. **Partitioning** by date or timestamp column — reduces bytes scanned dramatically. Daily partitions on `_loaded_at` for FDP, ingestion-time partitioning for ODP when no obvious column.
2. **Clustering** on high-cardinality, frequently-filtered columns (e.g., `customer_id`, `entity_id`). Reduces bytes for filtered queries.
3. **Column types** — choose tightest type that fits (`INT64` not `STRING` for numeric IDs; `DATE` not `TIMESTAMP` if no time needed). Smaller types compress better.
4. **NULL-aware schema** — nullable columns cost zero for nulls; required columns cost their type size.
5. **Active vs long-term storage pricing** — BigQuery automatically moves untouched data to long-term storage (half price) after 90 days. Don't artificially "touch" old partitions to keep them in active.
6. **Table expiration** for ephemeral data (sandbox / dev datasets — set a default 7-day table expiration so forgotten tables vanish).
7. **Avoid SELECT *** in views/materialised views — materialisation cost scales with column count.
8. **Snapshots / time travel** — be aware of the 7-day time-travel window cost; consider extending or shortening based on need.
9. **Compression awareness** — repeated/sorted columns compress better; reorder columns in CREATE TABLE if you control creation.
10. **Materialised views** for expensive aggregations that are queried often.
11. **Partition expiration** to auto-delete old partitions (e.g., 18-month retention on FDP).

A good answer covers partitioning, clustering, type choice, and active/long-term pricing.

An average answer mentions partitioning but nothing else.

Weak signals:

- Treating BigQuery like Postgres ("we'd add indexes" — BigQuery doesn't have indexes; you cluster instead).
- No understanding of bytes-scanned billing model.

Follow-up probes:

- "When does clustering not help?"
- "How would you investigate why a query suddenly costs 10× more?"
- "What's the difference between physical and logical storage billing?"

Section C — Data Engineering Best Practice

These three are the most senior — they test judgement, not just knowledge.

Q7. How would you prevent or manage data duplication in an FDP layer in BigQuery using dbt? Example: a pipeline accidentally ran twice.

> **Merged into Q3 for the 60-min interview.** This section remains in the guide as the deeper-dive answer key for the merged question — read it as the back half of Q3's expected answer.

What we're testing: *operational* duplication thinking — not just the SQL pattern, but what to do when it has already happened.

A great answer covers two horizons:

Prevention:

1. Same content as Q3 — incremental + `unique_key` + `merge` strategy + dbt `unique` test.
2. **Run-ID-aware materialisation.** Each dbt run carries an invocation ID; the model can deduplicate by `(business_key, _fdp_run_id)` so a re-run of the same upstream batch produces the same row, not a duplicate.
3. **Source-side idempotence.** The upstream ODP load itself should be idempotent — if Dataflow re-runs the same file, it produces the same ODP rows (via merge or pre-load deduplication). The FDP duplication problem often starts in the ODP.
4. **Airflow `max_active_runs=1`** on the relevant DAG so two pipeline runs cannot fire simultaneously.

Detection and recovery (when it happens anyway):

5. **Detect** via dbt's `unique` test failing after a run, or via a daily reconciliation query (`SELECT business_key, COUNT(*) FROM fdp.x GROUP BY 1 HAVING COUNT(*) > 1`).
6. **Recover** via a controlled deduplication SQL — keep the row with the latest `_fdp_run_id` or `_loaded_at`. Audit-log the deletion.

7. **Investigate root cause.** Audit trail tells you which run IDs generated duplicates. Was it a concurrent run? A schema mismatch in the source that caused the merge predicate to miss? An ingestion bug?
8. **Post-incident:** add a dbt test (uniqueness on business key) so the failure mode catches it next time, before it propagates to CDPs.

A good answer covers prevention plus a basic detection query and dedup recovery.

An average answer says "we'd just delete the duplicates" without explaining how to identify them or prevent it next time.

Weak signals:

- No mention of a `unique_key` strategy.
- No understanding of how the duplication propagates to downstream CDPs.
- Suggesting the fix is "ask the team to be more careful."

Follow-up probes:

- "Say the duplicate has already propagated into a CDP that another team owns — how do you handle the comms?"
- "Walk me through the SQL for the recovery dedup."
- "What test would you add to your dbt project to prevent it next time?"

Q8. Scenario — validating a bespoke CDP in production

Scenario to read to the candidate (verbatim):

> "For GDW we need to build a bespoke CDP. Imagine that CDP has been live in production for several months. **(a)** How would you validate the quality of the data in that CDP while it's running in production? **(b)** And separately, how would you validate the data items that the CJM has mapped — that is, confirm each mapped field is actually the *correct* field from the CDS inputs?"

Pause and let the candidate work through (a) first. Then pose (b).

What we're testing: real production data-quality thinking, not "we'd add some dbt tests". The interview-defining signal we're listening for is **using live source data to validate the CDP** — sampling real records, reconciling against the CDS inputs the CJM was mapped to. A strong candidate volunteers this without prompting; an average candidate has to be probed.

This is a scenario question with two distinct parts. Score them together but listen for distinct quality of thinking on each.

Part (a) — Validate the CDP's data quality while it's in production A great answer covers:

1. **Continuous monitoring, not one-off tests.** A live CDP cannot rely on pre-deploy dbt tests alone; the candidate frames this as ongoing surveillance.
2. **Profile-and-trend.** Track per-partition row counts, null rates, distinct value counts, distribution moments (mean / median / stdev for numerics) over a rolling baseline (e.g., 30 days). Alert on three-sigma drift, sudden cliffs, or zero-row partitions.
3. **Reconciliation against source.** For each CDP table, compute counts and key aggregates from the **CDS inputs** the CJM mapped, then compare. Daily reconciliation job that emits a pass/fail row to a `data_quality_runs` table.
4. **Sampling.** Pull N random records from the CDP each day, trace them back to the source via the CJM mapping, and assert fidelity. Bonus if they explain why sampling is needed (full re-validation is expensive, sampling gives statistical confidence).
5. **Standard quality dimensions still apply** — completeness, validity, uniqueness, referential integrity, timeliness — but now as **continuous probes**, not one-shot dbt tests. Some run via dbt as scheduled tests; some run as Cloud Scheduler / Composer / Airflow tasks; some are streaming dashboards.
6. **Anomaly detection.** Per-field statistical anomalies surfaced as alerts (Slack / PagerDuty), not just dashboards.
7. **Quality grade / score.** A composite signal (A–F or 0–100) computed daily so consumers know at a glance whether the CDP is healthy.
8. **Quality gates on downstream propagation.** If the CDP is feeding marts or other CDPs, the downstream consumers don't pick up a fresh CDP partition until its quality grade passes.

Part (a) good answer: items 1–4 and 6.

Part (a) average answer: "we'd add dbt tests" — true but missing the production-monitoring framing. Probe: "What if the test passes at build time but the data drifts a month later — how do you catch that?"

Part (b) — Validate the CJM-mapped fields are the *correct* field This is the more senior part. It's not about "is the value of the right *type*", it's about "is the value the right *thing*". The CJM has said "this CDP field comes from CDS column X" — is that mapping semantically right?

A great answer covers:

1. **Live-data round-trip validation.** Pick a real record in the CDP; pull the same business entity from the live CDS source; compare field by field. If `customer_full_name` in the CDP matches `customer_full_name` for the same customer in the CDS, the mapping is right. If not, either the mapping is wrong or the data has drifted. **This is the answer we are most listening for.**
2. **Business-meaning checks beyond type-checking.** A field typed `STRING` could be a name, an address, or anything. Validate the *shape* and *content* of values against business expectations: postcodes look like postcodes (regex), IBANs validate via the ISO checksum, dates fall within plausible business ranges, gender codes map to the agreed enum.
3. **Cross-reference against a second source where one exists.** If the same

entity exists in another upstream system (a parallel CDS or a CJM-blessed reference table), compare both. Discrepancies are a signal that one of the mappings is wrong.

4. **Stewardship and signoff.** A senior candidate mentions that ultimately the *data steward* (or whoever owns the CJM) must sign off on the mapping; the validation pipeline produces evidence, the steward makes the call.
5. **Lineage-aware tooling.** Mention Dataplex Lineage / Marquez / dbt's exposures and source-freshness — anything that ties the CDP field back to the CJM-declared source so a mismatch is detectable.
6. **Sample-and-spot-check process.** A weekly or monthly mapping audit: sample 100 records per CDP field, manually verify a portion against the live CDS via a Looker / BI tool, automate the rest with field-shape checks. Results logged so the next audit is differential.

Part (b) good answer: items 1, 2 and 6. Critically, candidate mentions **using live data**.

Part (b) average answer: "I'd write dbt tests for the mapped field." This is necessary but not sufficient — it tests the *value* is plausible, not that the *mapping* is correct. Probe explicitly: *"How do you know the mapping itself is right, beyond the value being plausible?"*

Weak signals across both parts:

- Treating the question as "we'd run dbt test" and stopping there.
- No mention of live data anywhere.
- Suggesting that quality is checked once at build time and not again.
- No awareness that the CJM mapping itself could be wrong.
- Confusing **type validity** (the value parses as a date) with **semantic correctness** (it's the correct date for that business entity).

Follow-up probes (use if you have time):

- "What's your minimum quality grade for a CDP to be safe to release into production?"
- "Suppose the CJM-mapped field is correct in 99% of records but wrong in 1%. How would you detect that, and what would you do about it?"
- "What's your strategy if the CDS source schema changes — how does the CJM keep up?"
- "Who owns the alert when CDP quality drops in production — the team, the data steward, or you?"

Scoring guidance: the question is worth 5 points overall. Distribute roughly 2 points to (a), 3 points to (b) — (b) is the harder, more senior part. A candidate who only does (a) well caps at 3/5. A candidate who does both, with live-data validation called out, lands 4–5.

Scenario bank — *if you want to vary the scenario for the second candidate or for future rounds, swap the GDW context for one of these:*

- *"You inherit a CDP that the previous owner built. You have no documentation, no CJM. How do you assess its quality before you trust it?"*

- *"A downstream consumer says one of your CDP fields has been wrong for three weeks. Where do you start?"*
- *"You need to migrate a CDP from one source CDS to a replacement CDS without breaking downstream consumers. How do you validate the migration?"*

Q9. How would you implement a CI/CD pipeline for a pipeline that runs either dbt or Dataflow, on a technology of your choice?

What we're testing: end-to-end engineering — the hardest senior-engineer skill.

A great answer is structured by stage:

Source control & branching:

- Trunk-based or short-lived feature branches.
- PR-based code review with at least one approver from the data team.
- Branch protection — required CI checks, no direct main pushes.

CI on PR:

- **Lint** — `sqlfluff` for dbt SQL; `ruff` / `black` / `mypy` for Beam Python; `yamllint` for configs.
- **Unit tests** — `pytest` for Python code; `dbt parse` / `dbt compile` for dbt models.
- **dbt slim CI** — compile and run only changed models against a CI BigQuery dataset, not the whole project.
- **Integration test** — DirectRunner Beam tests with fake GCP clients; or a smoke run against a sandbox dataset.
- **Security** — SAST (Qodana, Snyk), dependency scanning.

Deploy on merge to main:

- **Build the Dataflow Flex Template image** (Cloud Build → Artifact Registry); tag with the git SHA.
- **Build the dbt artefact** — package the dbt project (manifest, models, macros) and upload to GCS or a versioned location.
- **Update Airflow DAGs** if the orchestration changed (gsutil rsync to the Composer DAG bucket).
- **Run a smoke test** in a staging GCP project — small fixture, full E2E, must pass.
- **Promote to production** via a controlled mechanism: tagging the image with `prod`, updating the Airflow Variable that pins the Flex Template version, or via a manual gate (`gh workflow run`).

Auth:

- **Workload Identity Federation** from GitHub Actions (or whatever CI) — no JSON keys.
- Per-environment service accounts; CI uses the deploy-environment SA, not prod SA.

Environments:

- **Dev** — feature branches deploy to a developer sandbox.

- **Staging** — main branch auto-deploys to staging; smoke test runs.
- **Prod** — manual promote OR auto-promote after staging passes; rollback via image tag pinning.

Observability of the CI pipeline itself:

- Failure alerts to Slack.
- Test result artefacts archived per run.
- Deploy logs captured.

Rollback:

- For Dataflow: pin the prior Flex Template image tag.
- For dbt: re-deploy the prior dbt artefact, possibly with `dbt run --target=prod --full-refresh` on affected models.
- For Airflow: re-deploy the prior DAG version.

A good answer covers source control + PR CI + Flex Template build + deploy with WIF.

An average answer mentions "we'd use GitHub Actions" without explaining what each stage tests or how rollback works.

Weak signals:

- No mention of separate dev/staging/prod.
- No mention of how rollback works.
- Suggesting committing JSON keys.
- No idea what dbt slim CI is.

Follow-up probes:

- "Walk me through what runs on a PR vs what runs on merge to main."
- "How do you keep CI cheap? PRs trigger expensive Dataflow jobs by default — how do you avoid that?"
- "What's your strategy if the prod deploy succeeds but the smoke test fails?"
- "How does a dbt schema change get rolled back?"

Conversational / collaboration probes (PO leads, ~5 min)

These aren't scored on the rubric but the PO should note culture-fit signals:

- "Tell me about a time you disagreed with another senior engineer on architecture. How did you resolve it?"
- "How do you handle a request from another team to change a schema you own with two days' notice?"
- "What's the worst pipeline you've ever inherited, and what did you do about it?"
- "How do you keep up with the GCP ecosystem given how fast it moves?"

Score sheet

Score each question 1–5. Add notes per candidate. Calibrate after both candidates are done.

Primary questions (asked live in the 60-minute slot):

#	Question	Ca
1	ODP/FDP from fixed-width GCS file (+ “no golden path” probe)	
2	Data remediation steps	
3	Prevent **and recover from** duplication in dbt FDP	
5	dbt auth dev vs prod	
8	Scenario: validating a bespoke GDW CDP in production (2-part)	
9	CI/CD for dbt or Dataflow	
Primary total (out of 30)		

Soft skills (1–5 each):

- **Collaboration / communication** (PO judges): | A: | B: |
- **Coachability / curiosity** (peer engineer judges): | A: | B: |
- **Domain alignment** (Joseph judges): | A: | B: |
- **Soft skills subtotal (out of 15)** | A: | B: |

Grand total (out of 45): | A: | B: |

Banding (out of 45 total):

- **37+ (~82%) — strong hire.** Senior-engineer signal across every category. Bring on.
- **31–36 (~70–80%) — hire.** Some gaps but the architecture and operational thinking are there.
- **25–30 (~55–65%) — borderline.** Discuss as a panel. May indicate strong in some areas and weak in others — decide whether the gaps are coachable.
- **<25 — no hire** at E-level, but possibly reconsider for a more junior role with mentoring.

Optional / follow-up bank (only score if you actually asked them):

#	Question	Candidate A	Candidate B
4	Methods to run Dataflow		
6	BigQuery storage optimisation		
7	(already merged into Q3)	—	—

Decision matrix

After scoring both candidates:

1. **Both above 37:** hire both. Confirm headcount supports it.

2. **One above 37, one below 31:** hire the strong one. Give clear feedback to the other; if internal candidate, suggest a focused dev plan with concrete next-step skills (e.g., "build a Flex Template end-to-end in your current role; revisit in 6 months").
3. **Both 31–36:** discuss as a panel. Pick the candidate with the stronger collaboration / coachability scores, since gaps are addressable through mentoring. Or recommend a follow-up interview focused on the weakest area.
4. **Both below 31:** do not hire. Reset the hiring bar conversation with the leadership team.

Panel debrief format (5 min after each candidate)

Each panellist gives a **30-second take** in this order:

1. **Joseph:** technical depth — did they show E-level architectural thinking?
2. **Peer engineer:** would I want to work with them on a tough on-call rotation?
3. **PO:** can they communicate technical decisions to non-engineers?

Then scores. Then a single sentence per panellist: "Hire / no hire / discuss".

Post-interview comms

- **Within 24 hours:** decision communicated to candidate.
- **For internal candidates not progressing:** Joseph schedules a 30-minute follow-up to give specific, written feedback within one week. Frame it as a dev plan, not a rejection.
- **For successful candidates:** start-date conversation, project context, onboarding plan (which entity will be their first FDP build, who their buddy is, what their first month looks like).

Useful background reading we can point candidates at

If you want to share preparation material with the candidates, recommend:

- The team's internal architecture doc (whichever is canonical).
- Apache Beam programming guide — focus on side outputs and idempotent writes.
- The dbt docs page on incremental models — particularly the `merge` strategy.
- The Google Cloud Architecture Centre's data-lifecycle reference architectures.

Internally, the team's architecture documentation covers the three-unit deployment model, Beam ingestion patterns, dbt JOIN/MAP conventions, and observability / FinOps / governance practices that directly correspond to the questions above. Point candidates at the relevant internal pages if useful.

Last updated: 15 May 2026. Owner: Joseph Aruja (Engineering Lead).